Entrée unique de Gtion_EtatGlobal
🔗 link in : EVT_RAW_IN
Format minimal obligatoire des msg entrants :
msg = {
source: "UI" | "VPIV" | "SE" | "SYSTEM" | "JOYSTICK",
eventType: "VAR_CHANGE" | "CMD" | "INFO" | "ERROR",
payload: {
var: "CfgS",
prop: "modeRZ",
inst: "*",
val: "VEILLE"
}
}
🔗 Sorties officielles

VPIV_OUT
vers Arduino
vers SE

UI_OUT
affichage
activation / désactivation groupes
feedback utilisateur

EtatGlobalChanged
debug
logs
UI_Admin
historisation

👉 Ces sorties vont vers RZ_GestionFlux, pas directement ailleurs

Étape	
SF_Init_1	Chargement enrichedVars + valeurs par défaut
SF_Init_2	Détection / proposition IP (SP / SE / Arduino)
SF_Init_3	Initialisation UI Accueil (contexte INIT)
SF_Init_4	Initialisation VPIV Arduino + handshake
↓	Émission événement système INIT_OK

Gtion_Init-V1
mise en état

`RZ_GestionFlux` :

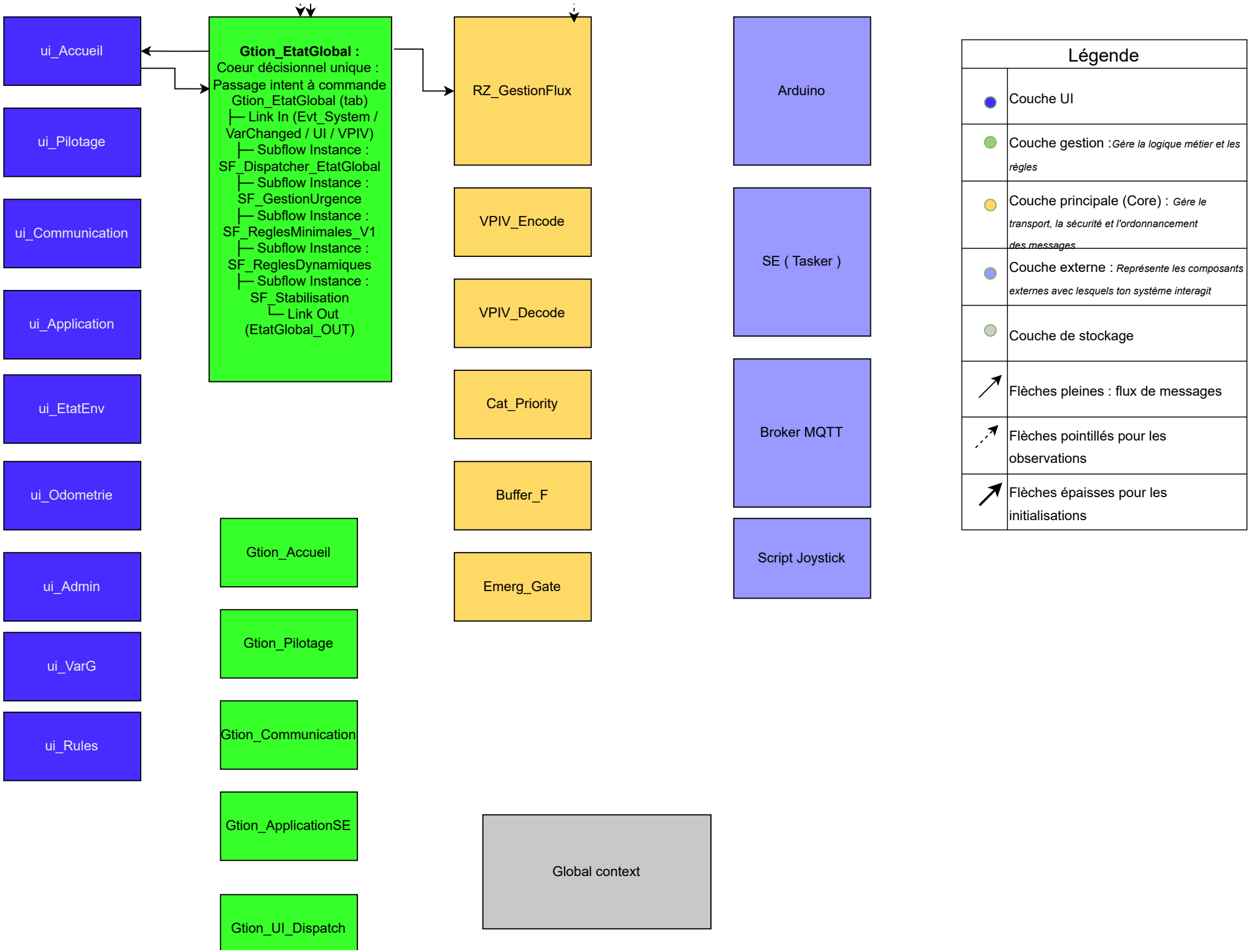
- encode VPIV
- choisit le topic (`PtoA`, `PtoSE`, etc.)
- publie MQTT
- met à jour certaines variables globales techniques
- **Il ne bloque jamais une décision validée**
- **Il n'interprète plus les règles métier**

- Entrées de `RZ_GestionFlux` :
- MQTT IN (Arduino → SP)
- **MQTT IN (SE → SP)**
- I events (boutons, menus, sliders)
- Joystick physique
  System events (startup, shutdown)
- 👉 **Toutes ces entrées convergent vers UN point logique** :

🔗 `link out : EVT_RAW_IN`

Après le branchement :

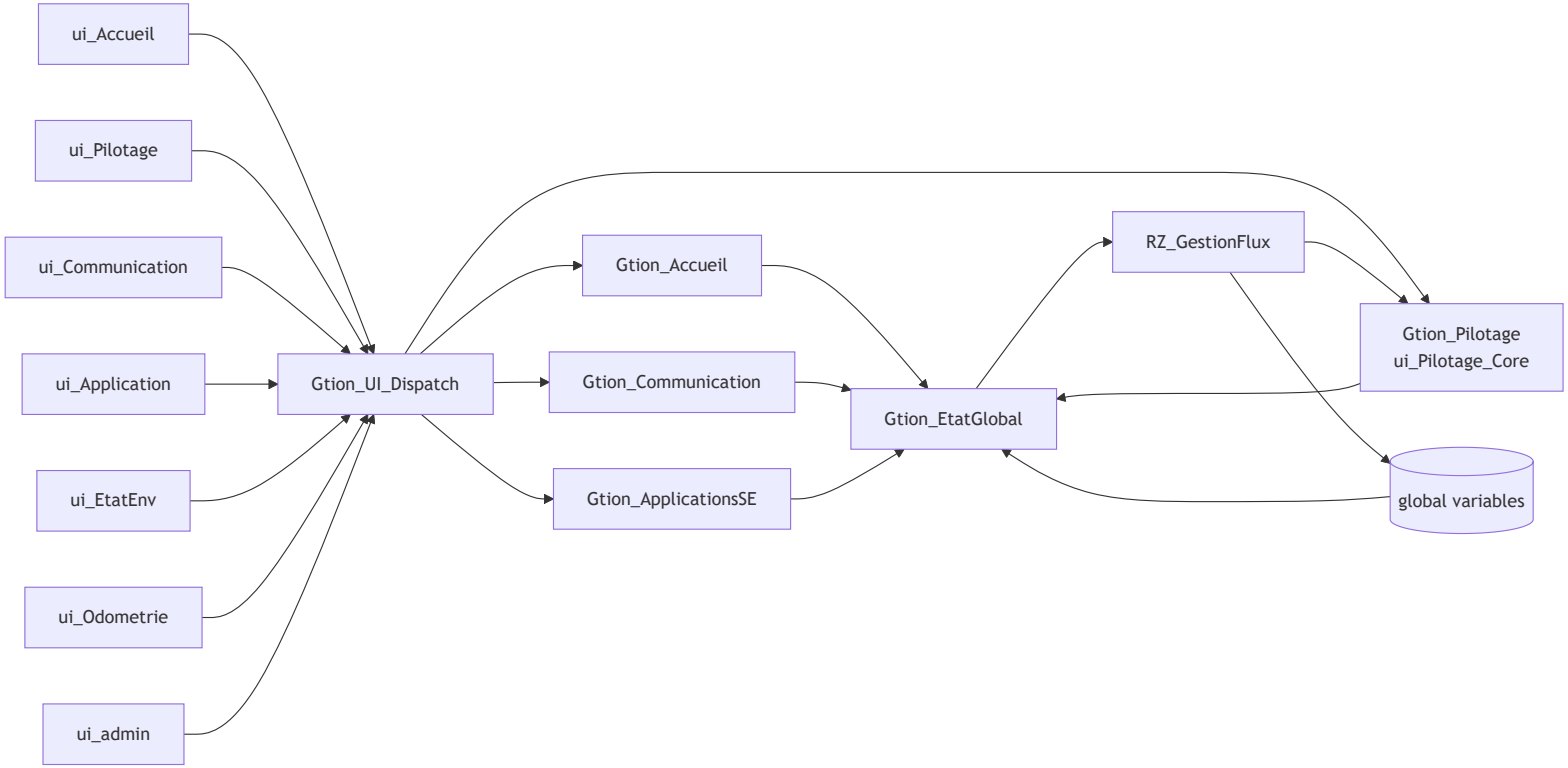
- `RZ_GestionFlux` :
  - encode VPIP
  - choisit le topic (`PtoA`, `PtoSE`, etc.)
  - publie MQTT
  - met à jour certaines variables globales techniques
- **Il ne bloque jamais une décision validée**
- **Il n'interprète plus les règles métier**


ui_Accueil
ui_Pilotage
ui_Communication
ui_Application
ui_EtatEnv
ui_Odometrie
ui_Admin
ui_VarG
ui_Rules
Gtion_EtatGlobal :
Coeur décisionnel unique :
Passage intent à commande
Gtion_EtatGlobal (tab)
├─ Link In (Evt_System / VarChanged / UI / VPIV)
├─ Subflow Instance : SF_Dispatcher_EtatGlobal
├─ Subflow Instance : SF_GestionUrgence
├─ Subflow Instance : SF_ReglesMinimales_V1
├─ Subflow Instance : SF_ReglesDynamiques
├─ Subflow Instance : SF_Stabilisation
└─ Link Out (EtatGlobal_OUT)
Gtion_Accueil
Gtion_Pilotage
Gtion_Communication
Gtion_ApplicationSE
Gtion_UI_Dispatch
RZ_GestionFlux
VPIV_Encode
VPIV_Decode
Cat_Priority
Buffer_F
Emerg_Gate
Global context
Arduino
SE (Tasker)
Broker MQTT
Script Joystick
Légende
Couche UI
Couche gestion : Gère la logique métier et les règles
Couche principale (Core) : Gère le transport, la sécurité et l'ordonnancement des messages
Couche externe : Représente les composants externes avec lesquels ton système interagit
Couche de stockage
Flèches pleines : flux de messages
Flèches pointillés pour les observations
Flèches épaisses pour les initialisations

Fichier JSON dictionnaire

Fichier JSON valeurs persistantes

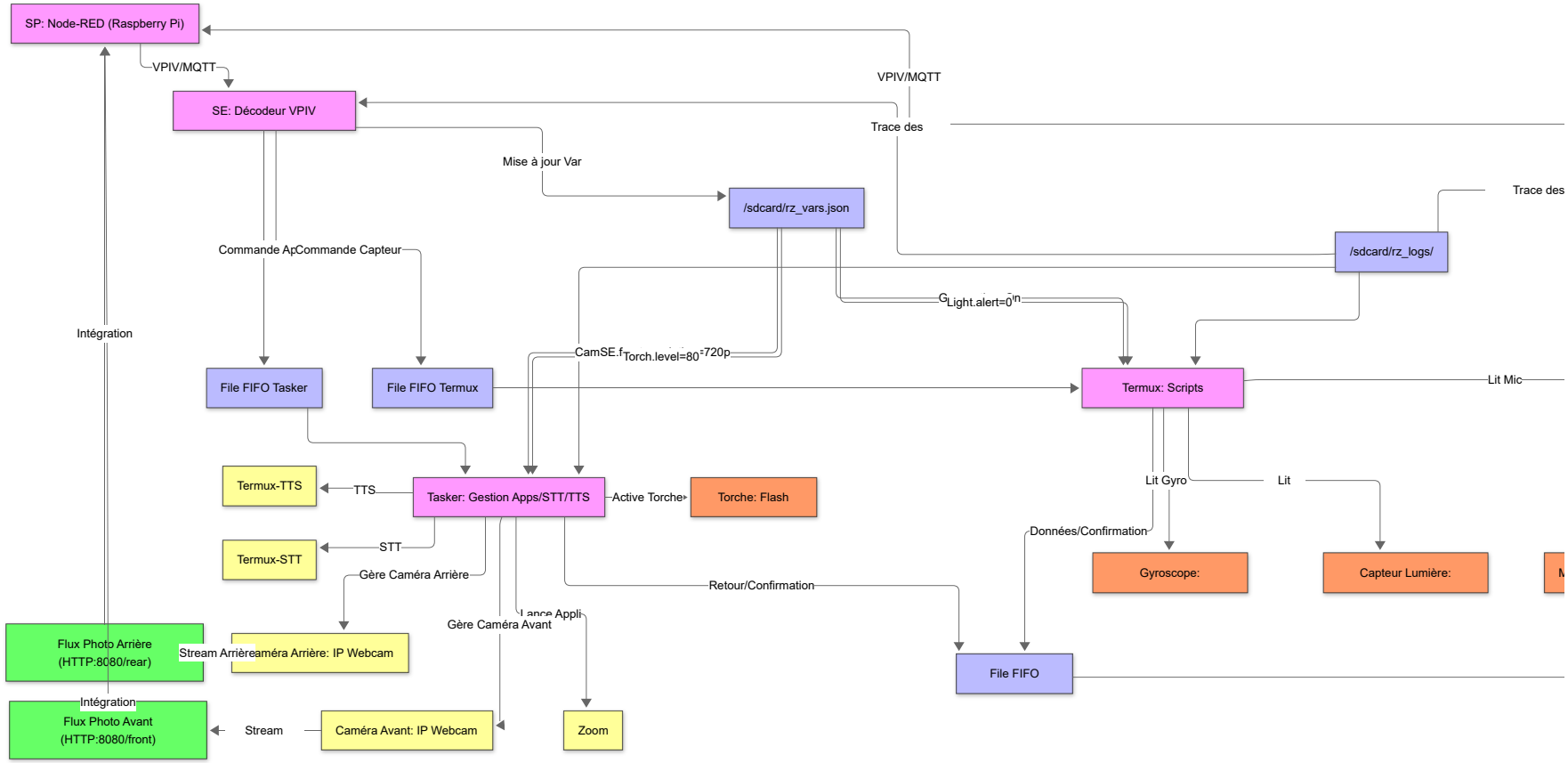
Utilisateur
↓
UI (joystick / écran)
↓ (intent éphémère)
Validation (Gtion_EtatGlobal)
↓
Commande VPIV (A ou V)
↓
MQTT
↓
SE / Arduino
↓
Action réelle
↓
VPIV retour (I ou E)
↓
SP
↓
Mise à jour variables globales
↓
UI observe l'état

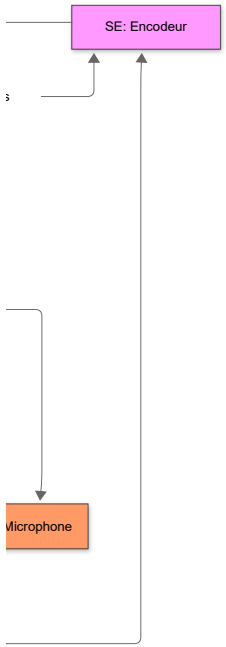


- **Pages UI** (à gauche) qui redirigent vers un **Dispatch UI**
- Le **Dispatch** distribue les actions vers différents modules de gestion
- Le **module de pilotage** (GPilot) interagit avec le **noyau** (RZ)
- Tous les modules de gestion convergent vers **l'état global** (GEtat)
- L'état global alimente le noyau qui gère les **variables globales**

- L'état global alimente le noyau qui gère les **variables globales**
- Les variables globales influencent à leur tour l'état global

Cela ressemble à une architecture de type MVC/MVVM avec un dispatcher central et un gestionnaire d'état global.te




SE: Encodeur
Microphone

Élément	Type	Description	Exemple de Message VPIV
Décodeur VPIV	Composant	Parse les messages VPIV et les achemine vers les files FIFO.	$A:Appli:Zoom:*:["open"]#
FIFO_Tasker	File	Canal pour les commandes applicatives (Zoom, TTS, Torche, Caméras).	$A:CamSE:stream:front:stream#
FIFO_Termux	File	Canal pour les commandes capteurs (Gyro, Light, Mic).	$V:Gyro:ParaM:*:{...}#
FIFO_Return	File	Canal pour les retours/confirmations.	$I:Torch:level:*:80#
VarJSON	Fichier	Stocke les paramètres des composants (ex: Gyro.active=On).	/sdcard/rz_vars.json
Logs	Dossier	Trace tous les échanges et erreurs.	/sdcard/rz_logs/parse.log
Gyro/Light/Mic	Capteur	Périphériques physiques (Gyroscope, Lumière, Microphone).	$F:Gyro:xyz:*:[120,-45,980]#
Torch	Actionneur	Torche (flash LED) du smartphone.	$F:Torch:level:*:80#
CamFront/CamRear	Caméra	Caméras avant/arrière (IP Webcam).	$I:CamSE:stream:front:stream#
StreamFront/StreamRear	Flux	Flux vidéo HTTP des caméras.	http://SE:8080/front
Zoom/TTS/STT	Application	Applications tierces (Zoom, Synthèse Vocale, Reconnaissance Vocale).	$A:Appli:Zoom:*:["open"]#

```
scripts_RZ_SE/
├── termux/
│   ├── rz_parse_vpiv_fifo.sh      # Décodeur VPIV → Files FIFO
│   ├── rz_tasker_listener.sh      # Écouteur Tasker
│   ├── rz_termux_listener.sh      # Écouteur Termux (capteurs)
│   ├── rz_encode_vpiv.sh          # Encodeur VPIV
│   ├── rz_read_sensor.sh          # Lecture des capteurs
│   ├── rz_stt_processor.sh        # Traitement STT
│   └── start_se.sh                # Script de démarrage
```